

CprE 308 Homework 2

Department of Electrical and Computer Engineering
Iowa State University

Student Name: Gurumanie Singh Dhiman

University-ID: 911192340

Problem 1. Address translation (5 points)

Suppose we are given a virtual address 16390. If we are using paging and the page size is 4KB, what are the *virtual page number* and *offset* of the given virtual address? Suppose virtual page numbers start with 0.

Note: $2^{10}=1024$, $2^{11}=2048$, $2^{12}=4096$, $2^{13}=8192$, $2^{14}=16384$, $2^{15}=32768$, $2^{16}=65536$.

Page size = 4KB = 4096 = 2^{12}

To find the Virtual Page Number and Offset:

Since we know page size is 2^{12} , we convert the virtual address 16390 to hex first to find the VPN and Offset.

$16390 = 0x4006$

With the above, the last 12 bits are the **VPN: 0x006 = 6** (Decimal)

the rest of the bits are the **Offset: 0x4 = 4** (Decimal)

Problem 2. Multi-level page table (5 points)

Suppose a program has a 64-bit virtual address space, and the system uses a three-level page table. The virtual address has a 10-bit top level page table field (i.e., index for page global directory), 16-bit second level page table field (i.e., index for page middle directory), and 22-bit third level page table field, and an offset.

a) What is the page size?

Adding $10 + 16 + 22 = 48$ bits total, then 64 bit address space – 48 bits = 16 bits for offset.

$2^{16} = 65,536$ bytes = **64KB is the page size.**

b) What is the max number of virtual pages for this program?

Since we found in part (a) that the total number of VPN bits is 48, the **max number of virtual pages is 2^{48} .**

Problem 3. Page Replacement Algorithms (10 points)

Suppose the physical memory has four page frames and they are occupied by four pages (0 - 3). The following table shows the timestamp when each page is loaded to the frame (i.e., the “Loaded” column), the timestamp when each page is last referenced (i.e., the “Last Ref.” column), the recently referenced bit R in the corresponding page table entry.

Page	Loaded	Last Ref.	R
0	126	280	0
1	230	265	0
2	140	270	1
3	110	285	1

Suppose a page fault happens and the system needs to replace a page.

- a) If the system uses First-In-First-Out (FIFO) algorithm, which page is replaced?

Page 3 is replaced based on First-In-First-Out (FIFO). This is further emphasised by the fact that page 3 was loaded at 110 whereas everything else was loaded much later. Even though it has been referenced most recently, based on FIFO it will be replaced first since it was the first to arrive.

- b) If the system uses Least-Recently-Used (LRU) algorithm, which page is replaced?

Page 1 is replaced based on Least-Recently-Used algorithm. This is because page 0 was last referenced at 280, page 1 was last referenced at 265, page 2 was last referenced at 270, and page 3 was last referenced at 285. Since page 0 was last referenced long ago compared to all other pages, it will be replaced. The recently referenced bit R is not used in this situation.

- c) If the system uses Clock algorithm, and the pages are organized in a circular list of 0-->1-->2-->3->0, the clock hand currently points to page 1, which page is replaced?

If the clock hand is currently pointing to page 1 then **page 1 is replaced**. This is done by checking the recently referenced bit R. In this case, since the clock hand was pointing at page 1 and the R bit was 0, it is replaced. If the clock hand pointed at a page that had its R bit set to 1, it would set it to 0 then move onto the next page to see if its R bit is set to 0 and then replace that.

- d) If the system uses Clock algorithm, and the pages are organized in a circular list of 0-->1-->2-->3->0, the clock hand currently points to page 2, which page is replaced?

If the clock hand is currently pointing to page 2 then **page 0 is replaced**. This is because since the clock hand starts checking from page 2, its R bit is set to 1 (so it sets it to 0), moves on. Then we check page 3, its R bit is set to 1 (so it sets it to 0), moves on. Then we arrive at page 0, its R bit is set to 0, so it is replaced.